



Every line of IR, and the pass that put it there — the whole `opt` + `llc` pipeline, in one report you can read.

A static, self-contained HTML report over the stock LLVM 22 tools.
No patched LLVM. No server. No build step.

pop rsp;ret.

Rakshit Awasthi rakshitawasthi17@gmail.com · Nutanix

Manas Ghandat ghandatmanas@gmail.com

Gyanendra Banjare gyanendrabanjare8@gmail.com · Abacus.ai

Chakilam Lalith Sai lalithsaichakilam3@gmail.com · IIT Roorkee

github.com/manasghandat/LLVM-Lens

01 Abstract

LLVM-Lens answers a question LLVM makes surprisingly hard to answer: *what did each pass in the pipeline actually do to my code?*

It runs the stock LLVM 22 tools with their own dump flags — `opt` for the middle end and `llc` for the backend — reads the stream they produce, and emits a single static HTML file. Open it in any browser. There is no server, no npm, no build step, and nothing to install on the machine that reads it.

What you get is one card per pass, holding every run that pass made: the before-and-after diff of any one of those runs and nothing else, the control-flow graph on either side of it, its timing, the analyses it ran, and the spill or register-allocation tables it produced if it was a backend pass. Every line of IR and Machine IR in the report carries the pass that wrote it, so you can click any line and walk its whole history back to the input. A flow view shows the entire lane at once, with runs that changed nothing collapsed, so the shape of the pipeline is visible in one screen instead of forty thousand lines of stdout.

You can also load your own pass plugin and have it treated as a first-class citizen of the report, badged and diffed like any stock pass.

LLVM-Lens is our implementation of the brief's LLVM Pass Transformation Analyzer (LPTA).

02 Why we built it

We wanted a good way to visualize the optimization pipeline: every pass that runs, in order, and what each one did to the code.

LLVM will tell you almost everything you need. It just won't tell you in a form you can read.

`opt -print-after-all` on one translation unit of a real C++ file is tens of thousands of lines of IR, in order, with no way to ask it anything. `-print-changed` is better — it only dumps when a pass actually modified the IR — but it tells you *that* a pass changed something, never *what*: to see the change you diff two consecutive dumps yourself, by hand, in an editor. `-debug-pass-manager` gives you the order of the pipeline and the analyses each pass requested, but none of the content. `-time-passes` aggregates a pass's time over the entire lane, so when `SR0APass` ran twenty-eight times you get one number and no way to split it across the runs. And `llc` is a different world again: Machine IR, different flags, different structure, and no correspondence at all between its output and the IR you were just looking at.

So the workflow, for all of us, has been: dump everything to files, pick two, diff them, scroll, lose your place, do it again. It works. It is also why a "why is this loop not vectorized" question takes an hour.

The observation the whole tool rests on

In the second week we noticed something that changed the design. Because `opt` only dumps when a pass has *changed* the IR, the sequence of dumps is not a sequence of snapshots — it is a sequence of *edits*. A pass that produced no dump

produced no change. Which means every line in the final module has exactly one pass that put it there, not probably and not heuristically, but by construction.

That turns "which pass did this?" from a search into a lookup, and it is what makes the blame view well-defined rather than a guess. Everything else in LLVM-Lens serves that one property: if a line knows its author, you can ask the compiler questions instead of grepping at it.

The dumps are already a complete, ordered, line-level record of who changed what. Nobody had joined them up.

03 What we built

A Python package with one command, and a report that is just files.

```
$ llvm-lens examples/json-query/lexer.cpp -o report --passes 'default<02>'
$ open report/index.html
```

There is no custom LLVM pass here and no patched LLVM. LLVM-Lens drives the tools you already have and reads what they already print — a deliberate constraint, because a tool that needs its own LLVM build is a tool nobody runs on a Tuesday afternoon when they actually have a regression to chase.

source	a .c, .cpp, .ll or .bc file
opt	-print-changed=quiet -print-module-scope -debug-pass=manager -time-passes
llc	-print-after-all -debug-pass=Structure -time-passes
parsers	states, timelines, the blame walk, ISel and asm correlation
report	index.html + manifest.json + one chunk of IR per pass

lane A — middle end, LLVM IR · lane B — backend, Machine IR · the report output is static files

Two lanes. **Lane A** is the `opt` new-pass-manager pipeline over LLVM IR; **Lane B** is the `llc` backend over Machine IR. They stay two tabs but join into one timeline: a synthetic *Input IR* card opens lane A, an *Optimized MIR* card closes lane B, and a dashed edge marks the handoff between them. Read only the middle end, or start at the handoff and read codegen.

What it is made of

PART	LINES	WHAT IT DOES
src/llvm_lens/	4,575	drives the tools, parses their streams, walks blame, emits the report
frontend/app.js	2,846	the report itself: ten views, no framework
frontend/style.css	838	the report's own palette and layout
tests/ (21 modules)	5,754	264 tests, 12 skipped — parsers, blame, ISel, and the frontend

54 commits between 29 August and 18 September 2026. Vendored: cytoscape.js and dagre, for the graphs. Everything else is the standard library and vanilla JS. Licence GPL-3.0.

It survives a bad run

If `opt` crashes or times out, the report still gets written with whatever was captured up to that point, and says so. A half-answer you can read beats an exception trace, especially when the thing you are investigating is a compiler bug.

04 The report, view by view

Every figure below is a real screenshot of the tool running on a real input, rebuilt immediately before this document was written. The numbers in the prose were read back out of those same report files, not typed from memory.

4.1 The workspace

One screen holds everything: the lane's passes down the left with a churn bar per row, a spine at the very top showing every pass in the lane as a tick sized by how much it changed, the selected pass's own functions below that, the panes in the middle, and a log and analysis drawer along the bottom. The coloured marks beside each pass name are that pass's line churn — green added, red removed — so the rail doubles as a density map of where the work happened.

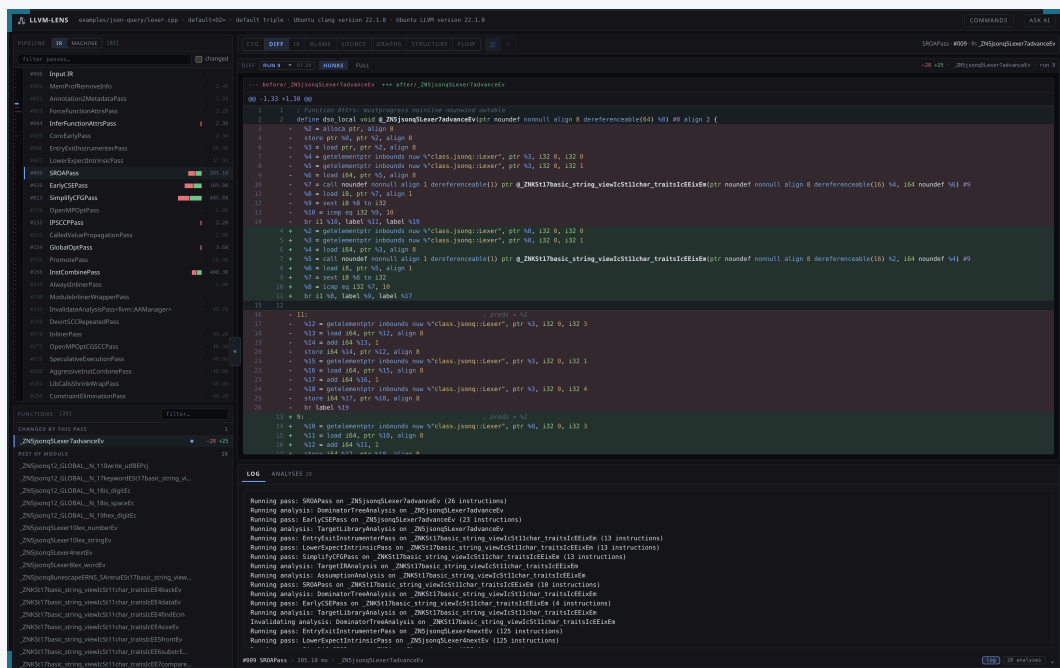


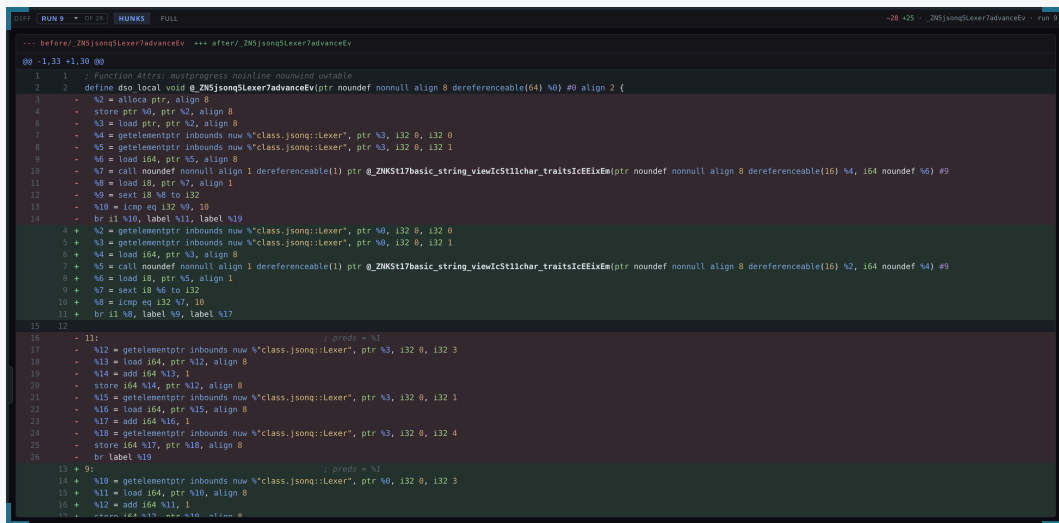
FIGURE 1 – THE WORKSPACE

SROAPass, card #009, run 9 of 28, on `Lexer::advance`. 168 cards in this report – 82 middle-end, 86 backend – covering 350 pass runs, and 22 of the middle-end cards changed something. A card is one pass however many times it ran: SROAPass is a single row in the rail for all 28 of its runs. The menu beside **DIFF** says which run the panes are reading, and it opens on the last one, because that is the state the card leaves behind. Card numbers are the lane position of the pass's *first* run. The drawer below says this run touched one function, `advance`.

4.2 Diff — one run, and nothing else

This is the view that replaces hand-diffing two dump files. It is the before-and-after of a single pass run, unified, with the churn counted in the header. The diff is exactly one run's work — never a pass's cumulative effect, and never the four other things that happened between two arbitrary dump points. The picker in front of it is what keeps that true once a pass has run many times: it names the run on screen, and the header repeats it.

Choosing an earlier run shows *that* run's diff, against the state it actually replaced. That is not the same as the state before the pass's previous run, because everything else in the lane ran in between — which is why a run that skipped a function has no diff to show for it and says so, pointing at the runs that did move it.



```
Diff [RUN 9] 100% HUNKS FULL -28 +25 -28 /_ZNSqSlexe7advanceEv - run 9
--- before/_ZNSqSlexe7advanceEv +++ after/_ZNSqSlexe7advanceEv
@@ -1,13 +1,30 @@
1 1 ; Function Attrs: mustprogress noinline nounwind optnone
2 2 define dso_local void @_ZNSqSlexe7advanceEv(ptr noundef nonnull align 8 dereferenceable(64) %0) #0 align 2 {
3 - %2 = alloca ptr, align 8
4 - store ptr %0, ptr %2, align 8
5 - %3 = load ptr, ptr %2, align 8
6 - %4 = getelementptr inbounds nuw %"class.jsong::lexer", ptr %3, i32 0, i32 0
7 - %5 = getelementptr inbounds nuw %"class.jsong::lexer", ptr %3, i32 0, i32 1
8 - %6 = load i64, ptr %5, align 8
9 - %7 = call noalias nonnull align 1 dereferenceable(1) @__ZNKSt17basic_string_viewIcSt11char_traitsIcEExEm(ptr noundef nonnull align 8 dereferenceable(16) %4, i64 noundef %6) #9
10 - %8 = load i8, ptr %7, align 1
11 - %9 = sext i8 %8 to i32
12 - %10 = icmp eq i32 %9, 10
13 - br i1 %10, label %11, label %19
14 + %2 = getelementptr inbounds nuw %"class.jsong::lexer", ptr %0, i32 0, i32 0
15 + %3 = getelementptr inbounds nuw %"class.jsong::lexer", ptr %0, i32 0, i32 1
16 + %4 = load i64, ptr %3, align 8
17 + %5 = call noalias nonnull align 1 dereferenceable(1) @__ZNKSt17basic_string_viewIcSt11char_traitsIcEExEm(ptr noundef nonnull align 8 dereferenceable(16) %4, i64 noundef %4) #9
18 + %6 = load i8, ptr %5, align 1
19 + %7 = sext i8 %6 to i32
20 + %8 = icmp eq i32 %7, 10
21 + br i1 %8, label %9, label %17
22
23 - ; preds = %1
24 - %12 = getelementptr inbounds nuw %"class.jsong::lexer", ptr %3, i32 0, i32 3
25 - %13 = load i64, ptr %12, align 8
26 - %14 = add i64 %13, 1
27 - store i64 %14, ptr %12, align 8
28 - %15 = getelementptr inbounds nuw %"class.jsong::lexer", ptr %3, i32 0, i32 1
29 - %16 = load i64, ptr %15, align 8
30 - %17 = add i64 %16, 1
31 - %18 = getelementptr inbounds nuw %"class.jsong::lexer", ptr %3, i32 0, i32 4
32 - store i64 %17, ptr %18, align 8
33 - br label %19
34 + ; preds = %1
35 + %10 = getelementptr inbounds nuw %"class.jsong::lexer", ptr %0, i32 0, i32 3
36 + %11 = load i64, ptr %10, align 8
37 + %12 = add i64 %11, 1
38 + store i64 %12, ptr %10, align 8
```

FIGURE 2 — THE DIFF, +25 -28

SROA doing what SROA does: `%2 = alloca ptr`, the `store` of `this` into it, and the `load` back out are all gone, and every later use now reads `%0` — the argument register — directly. The `getelementptr`s that follow change from `ptr %3` to `ptr %0`. Nothing else in the function moved. HUNKS and FULL switch between the changed regions and the entire function, so you can check that claim rather than trust it.

4.3 Blame — who wrote this line

Every line of IR knows the pass that last wrote it. The Blame view shows that as a gutter down the left: the ordinal of the pass run that produced the line, or `in` if the line has survived untouched since the input. Runs are tinted in eight rotating bands so a run's edits read as one block of colour rather than a scatter.

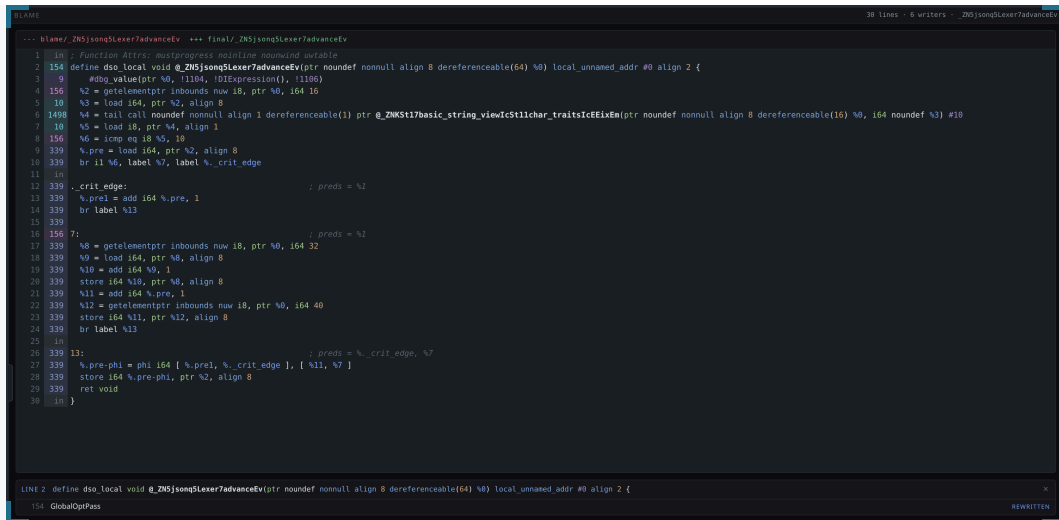


FIGURE 3 – THE GUTTER, AND ONE LINE'S WHOLE HISTORY

Lexeron::advance after the whole default<02> pipeline: 30 lines, 6 writers. The `in` rows are the lines no pass ever rewrote – carried through from the input. Everything else names its author, a run ordinal into the lane. Click any row and the lineage opens below it: every pass that touched that line since the input, oldest first, each tagged **created**, **rewritten** or **renamed**. That distinction is what makes the chain readable instead of a list of every pass in the lane: SSA renumbering turns `%3` into `%7` constantly, and that is not a rewrite – the instruction is the same instruction. Each entry jumps to that pass's card.

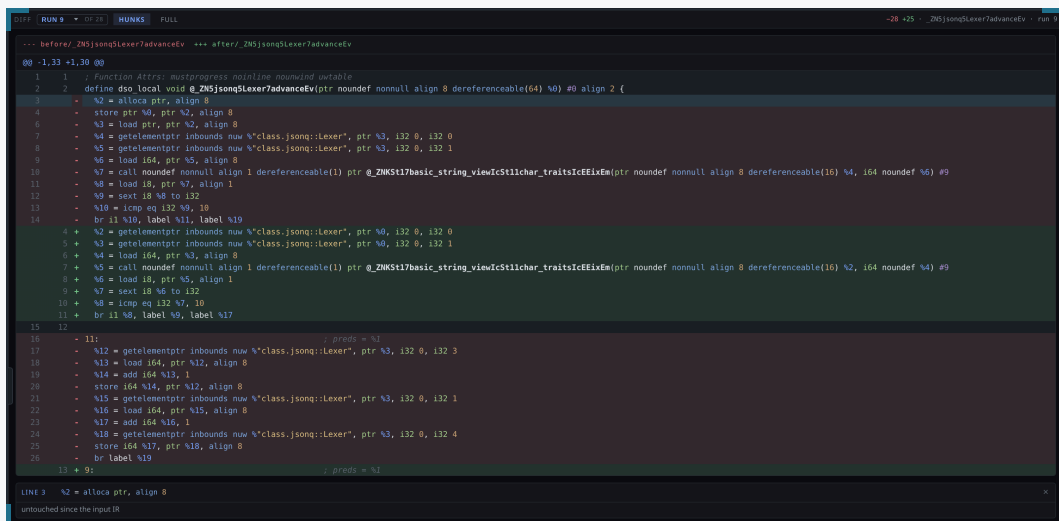


FIGURE 4 – THE QUESTION, ASKED WHERE IT COMES UP

Blame is reachable from the Diff view too, which is where you actually want it: this is the line SROA deleted, `%2 = alloca ptr, align 8`, and the answer is *untouched since the input IR* – it came from `clang -O0` and SROA is the first thing to touch it. Clicking a *green* line instead walks forward from wherever that line was created.

4.4 Flow – the shape of the pipeline

The diff view answers questions about one pass. The flow view answers the question you have before you know which pass to look at: *where in this pipeline is anything happening?* It lays the lane out as a serpentine, one node per pass, and collapses consecutive passes that changed nothing into a single `N` passes · unchanged span. Those collapsed spans are most of the pipeline, and hiding them is what makes the whole thing fit on a screen. Because a node is a pass rather than

a run, one node carries everything that pass did in the lane — `SROAPass` is one box holding all 28 of its runs, at its first run's ordinal.

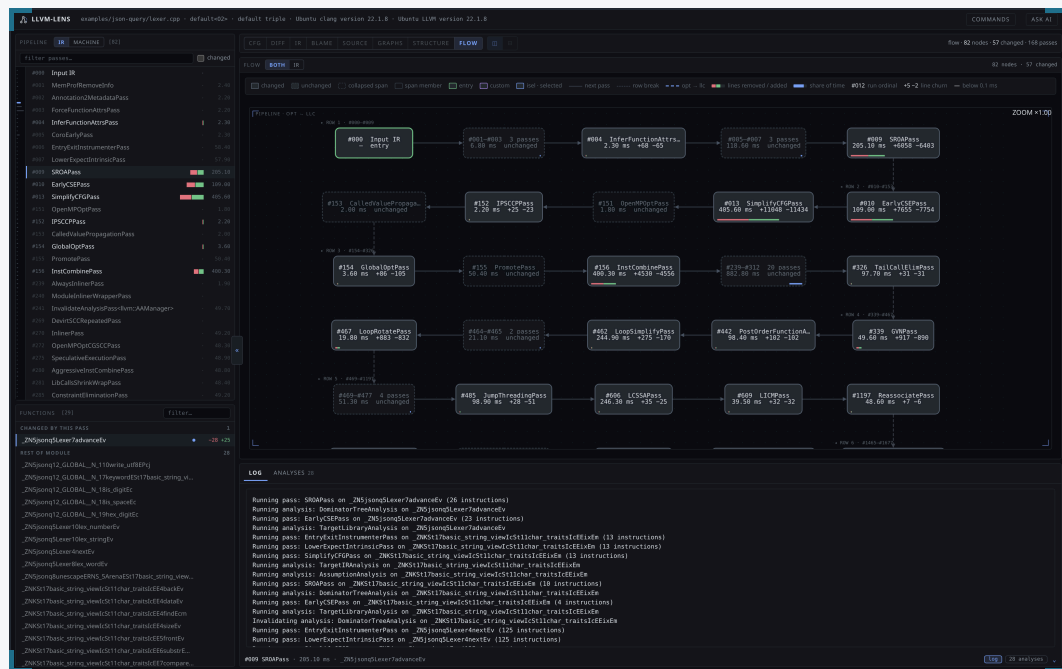


FIGURE 5 – BOTH LANES, 82 NODES, 57 CHANGED

Each node carries the ordinal of the pass's first run, its measured time, and its churn summed over every run it made; the bar along the bottom of a node is its share of the lane's total time. The dashed blue edge in the middle is the `opt` → `llc` handoff – the one place where LLVM IR becomes Machine IR. The legend across the top names every mark, including the purple *custom* node, which is where a loaded pass plugin shows up. A first pass down this view says where the work was: three of the first twenty nodes hold almost all of it, and the long tail is passes moving a few dozen lines each.

4.5 Structure — where in the pass manager

The new pass manager is a tree of nested managers, and which manager a pass lives in determines what it can see. Summarising per function and then bundling those into module-level numbers can hide that entirely. The structure view draws the real hierarchy – module, CGSCC, function, loop – with each row's analyses, invalidations and time, so a per-function pass and a module pass are visibly different things.

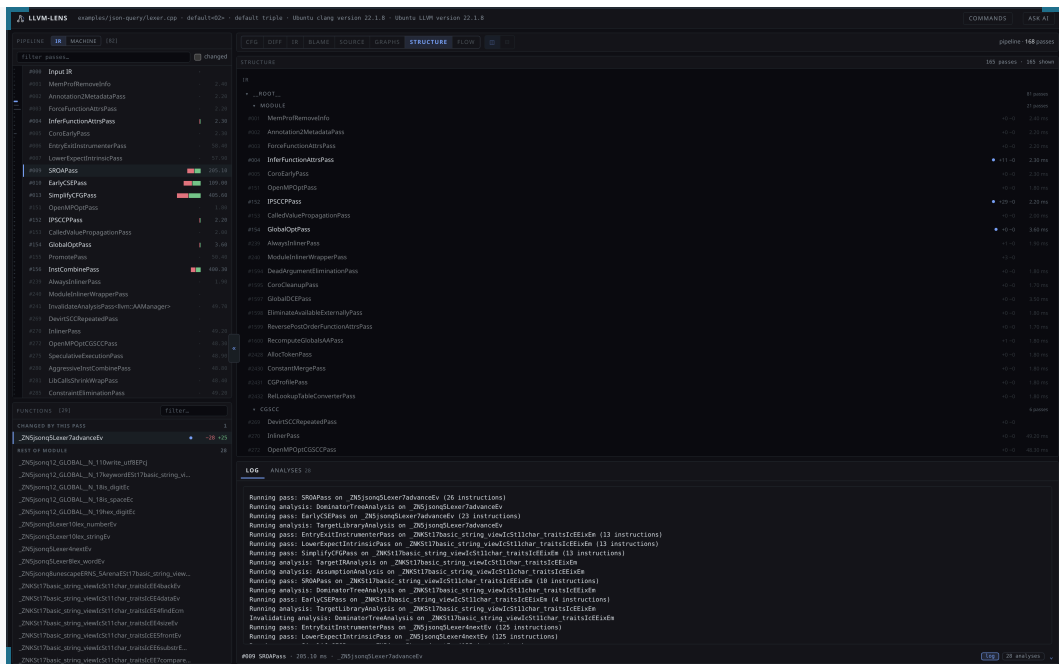


FIGURE 6 – THE HIERARCHY, 165 PASSES SHOWN

The tree is the pass manager's own nesting rather than a flat list – Module, CGSCC, Function and Loop for the middle end, the machine lane's module and function managers beneath, **Function 45 passes** and **Loop 9 passes** visible as their own groups. Every leaf carries the analyses it ran and the cached results it invalidated, then its time, and clicking a leaf opens that pass. Read the two columns together and the cost model shows up: **#004 InferFunctionAttrsPass** reads **+11 -0** at 2.30 ms – eleven analyses run, nothing thrown away – while **#002 Annotation2MetadataPass** reads **+0 -0** at 2.20 ms, so the second it costs buys nothing and forces no recomputation downstream.

4.6 Control-flow graphs, before and after

Each card carries a CFG of the selected function on both sides of the pass. Entry block outlined green, back edges red so loops are visible at a glance, block detail on selection. The zoom readout is live — the pane fits the graph, and you zoom from there.

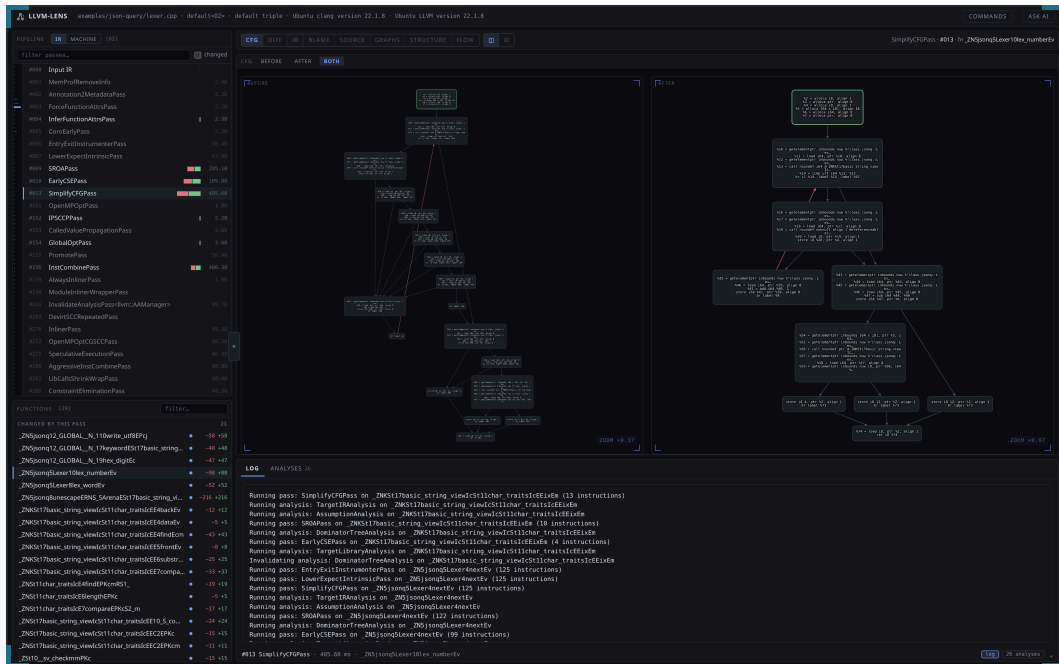


FIGURE 7 – SIMPLIFYCFG ON `LEXER::LEX_NUMBER`

Before, on the left: **18 blocks** at $\times 0.37$ zoom, a tangle of one- and two-instruction blocks falling out of the switch's comparison chain. After, on the right: **10 blocks** at $\times 0.67$. Same function, same pass, and the difference between the two pictures *is* the change – no diff reading required. The figure was taken on the run that actually did this – `SimplifyCFGPass`'s 43rd run, chosen in the diff pane – rather than on the card's default, which is its last; the capture script asserts the block count dropped before it takes the picture.

4.7 Analysis graphs

Dominator trees, control- and data-dependence graphs, post-dominators, loop nesting trees, and a module-wide call graph – computed from the module as it stands at the end of the pipeline. These are the views to reach for when the question is structural rather than textual: the dominance tree tells you what must execute before what, the dependence graph tells you what a value actually depends on, and the loop nest tree tells you what `LoopVectorizePass` was looking at. Being per-instruction, they only stay readable on a small function – the pane auto-fits, and on `Lexer::next` (77 blocks) it fits to $\times 0.18$ and reads as a smear. That is a real limit, and the report does not pretend otherwise.

4.8 Source – IR line back to C++ line

When the input carries debug info, every IR line maps back to the C++ line it came from, and the header says how many of them mapped. With inlining in play this is the only practical way to see which source a piece of IR actually belongs to.

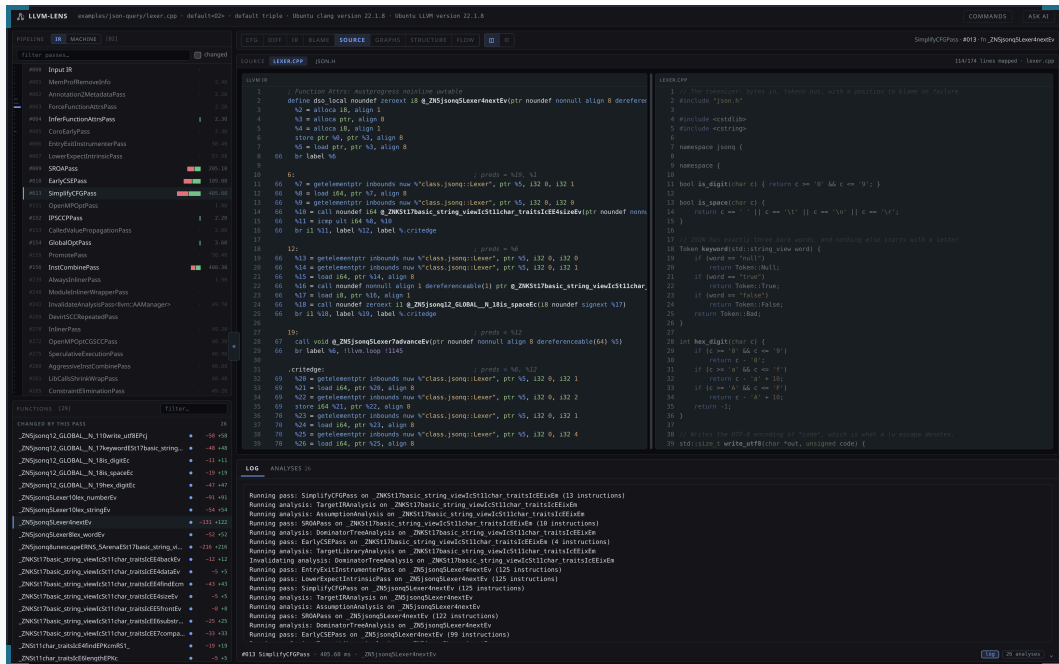


FIGURE 8 – 114/174 LINES MAPPED • LEXER.CPP, WITH THE FILE SWITCHER OPEN

The right pane is the real source; the highlighted lines are the ones this snapshot's IR points at. This is `Lexer::next` on `SimplifyCFGPass`'s 18th run, and the snapshot spans two files – `json.h` has been inlined into `lexer.cpp` – so a chip row appears above the source and you switch between them. The stat line follows the chip: it counts the lines mapped into the file you are looking at, not the total. The mapping only survives while the dump still carries the debug metadata: later in the pipeline `opt` has stripped it, and the pane says so in as many words – *no mapped lines for this function at this pass* – rather than showing an empty box that reads like a bug.

4.9 ISel and Asm – the two ends of the backend

The middle-end view is IR to IR. But the question "why is the generated code bad?" lives at the seam: which IR instructions became which machine instructions, and which machine instructions became which assembly lines.

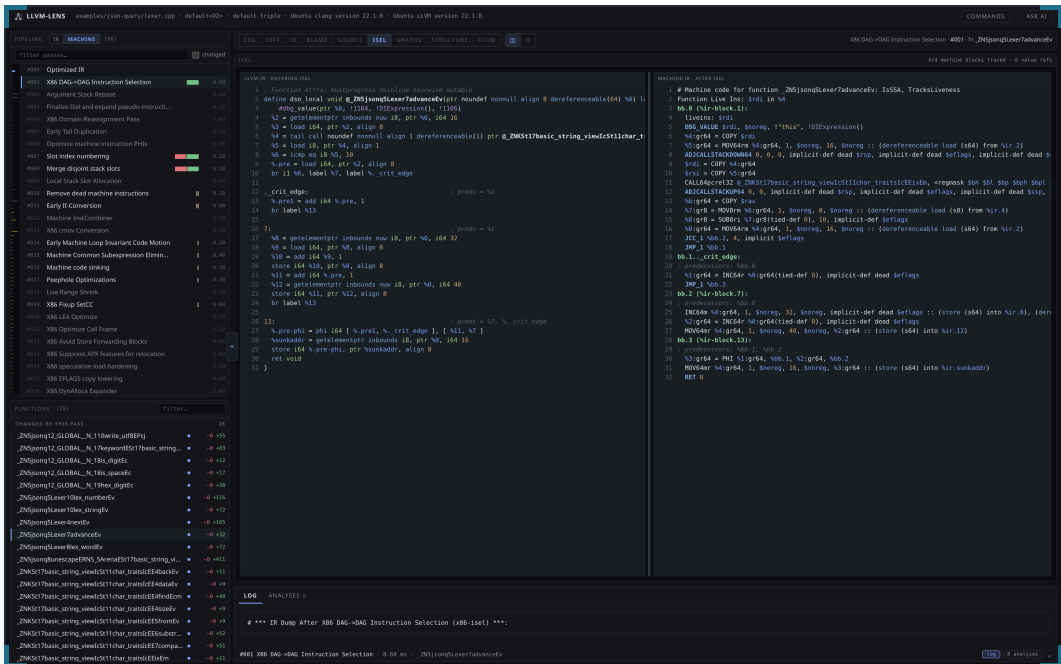


FIGURE 9 – INSTRUCTION SELECTION, 4/4 MACHINE BLOCKS TRACED • 6 VALUE REFS

Left, the IR entering instruction selection; right, the Machine IR leaving it. The pairing is not a guess about ordering – llc names each machine block after the IR block it came from (bb.2 (ir-block-7)), and value operands carry %ir.N references, so %7:gr8 = MOV8rm ... (dereferenceable load (s8) from %ir.4) really does point at a specific IR line. The header shows how much of the mapping was recovered, so you can tell a complete answer from a partial one.

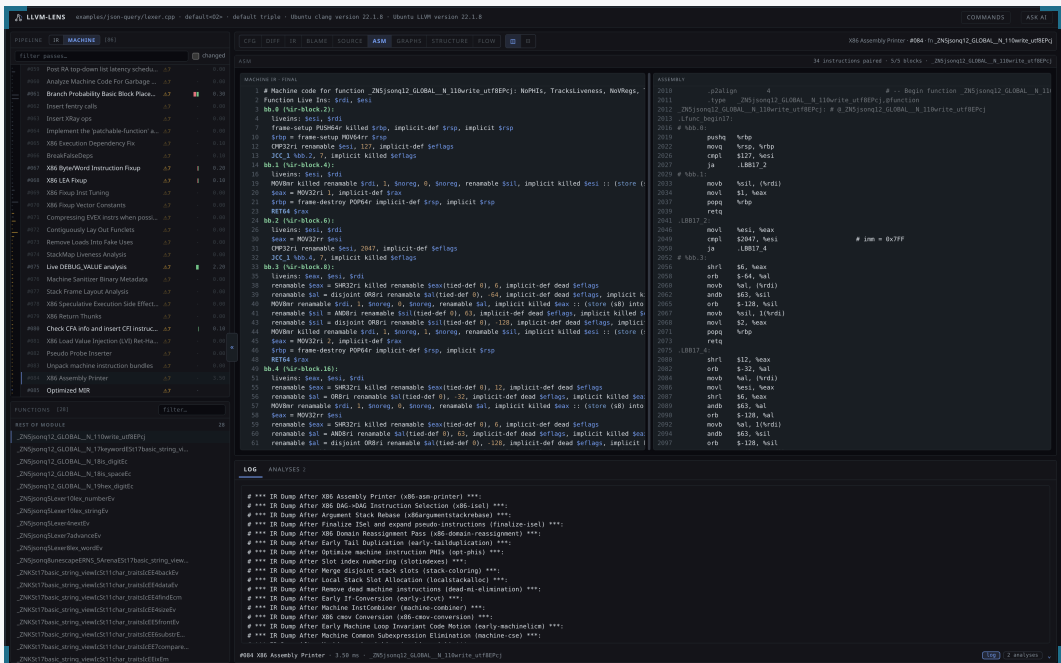


FIGURE 10 – FINAL MACHINE IR AGAINST THE ASSEMBLY PRINTED FROM IT

34 instructions paired • 5/5 blocks. The alignment is a longest common subsequence over an opcode-compatibility predicate – JCC_1 against ja, MOV64mr against movq – so it survives the assembler spelling things differently, and it reports how much it could not pair instead of silently dropping it.

4.10 Spills and registers

When the allocator runs out of registers it spills, and spilled code is usually the answer to "why is this slow". The backend passes carry their own drawers: a spill table naming every slot and the block it spilled in, and a register map from virtual register to physical register or stack slot, per function.

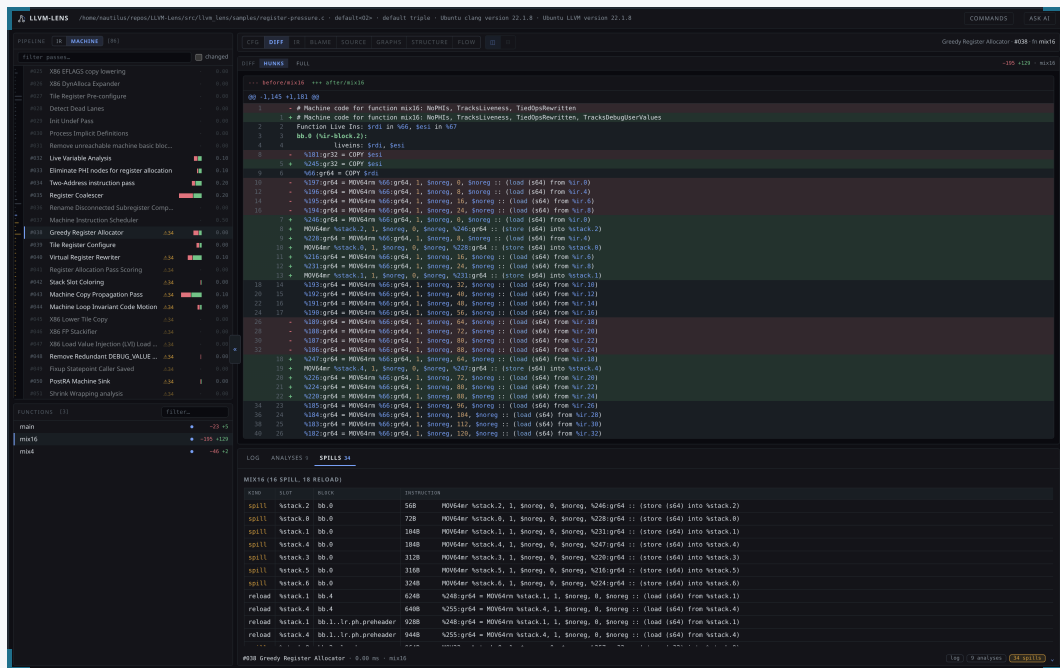


FIGURE 11 – MIX16 , 16 SPILLS AND 18 RELOADS

Greedy Register Allocator – the card is badged with the count (34) and so is every card downstream of it, because the count is a property of the function after allocation, not of one pass. The diff above shows `$gr64` virtual registers turning into `MOV64mr %stack.N, ...` stores; the Spills drawer below names the slot, the block (`bb.0`, `bb.4`, `bb.1..lr.ph.preheader`) and the exact instruction, which is the difference between knowing there are 34 spills and knowing where to look.

The companion drawer is the register map on Virtual Register Rewriter, the pass where virtual registers become real ones: for `main` it is nineteen rows, `%1` → `$rax`, `%5` → `$rax`, `%3` → `$noreg`. Switch the function to `mix4` in the same report and it is seven rows, every one a physical register, no stack slot anywhere. Same report, same pipeline, two outcomes – which is the whole reason that sample exists. Switch to the Spills drawer on `mix16` and the same contrast is there in the numbers: 34 spills against none.

4.11 Your own pass, as a first-class card

A tool for reading LLVM's pipeline is only half useful if it stops at the edge of the passes LLVM ships. Pass `--load-pass-plugin` and `--custom-pass` and your pass appears in the rail, in the flow, in the structure tree and in the blame chains, badged so it is never mistaken for a stock one. Because `opt` dumps custom passes less predictably than built-in ones, the tool forces a dump for them.

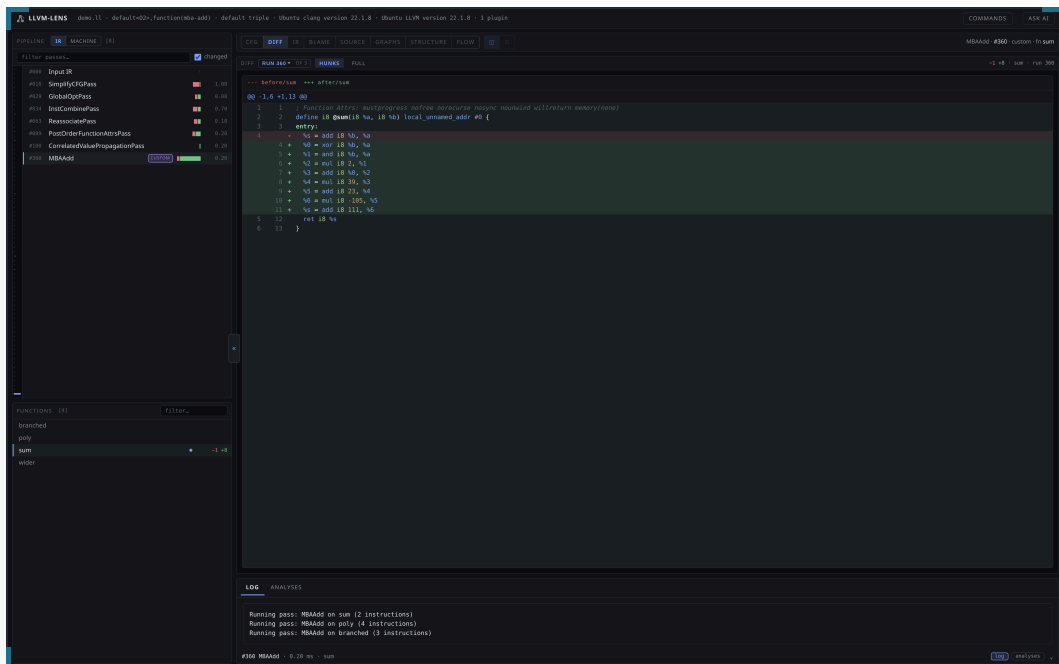


FIGURE 12 — A CUSTOM PASS, +8 -1, ON SUM

MBAAdd rewrites `a + b` into a mixed boolean-arithmetic equivalent: `%s = add i8 %b, %a` becomes eight instructions ending `%s = add i8 111, %b`. Same value, unrecognisable shape — the kind of transform the blame chain is there to survive. The header reads `1 plugin`, the rail badges the card `custom` in purple, and the context line reads `MBAAdd · #360 · custom`. Note the run number: each of its three runs rewrote a *different* function, and the card opens on its last, so this figure was taken on run 360 with the run picker. Only one of the four functions in the module changed in any one run; the pass is selective, and the card shows that rather than implying otherwise.

The same pass appears in the structure tree as `#360 MBAAdd`, badged `custom`, sitting inside `Function 5 passes` where a function pass belongs rather than alongside the module passes. And it is one purple node in the flow view — one node for the pass, however many runs it made, which is the same rule every other pass in that view follows. The plugin was wired entirely by a committed `llvm-lens.yml` beside the example, so running `llvm-lens demo.ll` in that directory needs no flags at all; the config file that did it is named in the report's own metadata, and it is worth a look on its own — [section 05](#) takes it apart.

4.12 Scale

One translation unit is the common case, but the tool is not limited to it. Link seven translation units of a real program into one module and the report grows to match — same views, same blame, 99 functions instead of 29, and the same 168 cards, because the pipeline is the same pipeline. What grows is inside them: the `SROAPass` card that holds 28 runs over one file holds 98 over the linked module, 830 runs in the lane against 264.

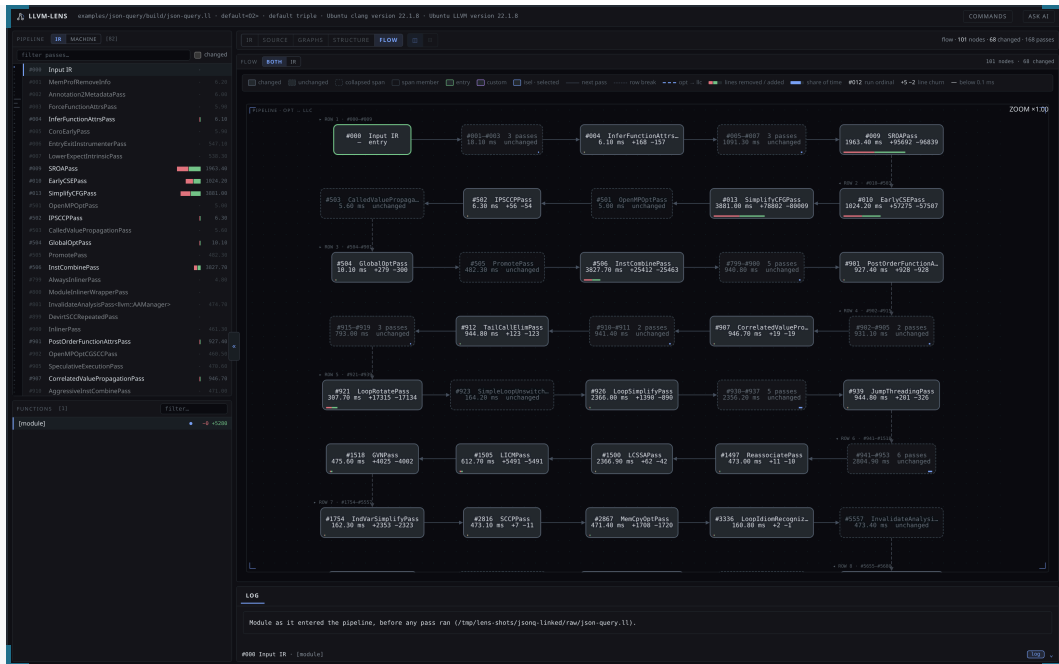


FIGURE 13 – ONE MODULE, SEVEN TRANSLATION UNITS, 101 FLOW NODES

The whole-program `default` pipeline, and the same 168 cards the single-TU report has: a bigger module does not add passes, it adds runs inside them, and the churn numbers are module-scope – `+95692 -96839` summed across `SROAPass`'s 98 runs – because at module level every function's edits land in one diff. Cost is real and we would rather say it than hide it: this report took about three minutes of tool time and writes 2.3 GB of intermediate dumps. The single-TU report of the same program is 21 seconds and 313 MB, and shows every feature the bigger one does.

4.13 Provenance — the report says how it was made

A report that is going to be attached to a bug or argued over in review needs to say what produced it. The commands sheet prints the exact argv of all three tool invocations, and the report carries the config file it read, the plugins it loaded, and the LLVM and clang versions.

A **commands** sheet in the report prints the verbatim argv of all three invocations, each with a copy button. This is the whole of what LLVM-Lens asks of LLVM, and it is what the report behind Figure 2 ran:

```
$ clang-22 -S -emit-llvm -O0 -Xclang -disable-O0-optnone -g \
  -o raw/lexer.ll examples/json-query/lexer.cpp

$ opt-22 -S -passes=default -print-changed=quiet -print-module-scope \
  -debug-pass-manager -time-passes -o raw/opt-final.ll raw/lexer.ll

$ llc-22 -print-after-all -debug-pass=Structure -time-passes \
  -o raw/final.s raw/opt-final.ll
```

Every report carries these, so a reader can reproduce an analysis exactly rather than approximately.

There is an optional **ask AI** panel that answers questions about the pass and function on screen, backed by your own Anthropic or an OpenAI-compatible key. It is off until you configure one, it sends only the selected pass's summary and a bounded window of mapped source, and the build flag `--no-ai` exists so that a

report you intend to share cannot carry a credential by accident. Every report in this document was built with `--no-ai`.

05 How it works

Four things are worth explaining properly: the blame walk, the two correlation layers, and the file that says how to build the report at all. The rest is parsing.

The blame walk

The dumps give us a sequence of states. Blame turns that sequence into per-line history, and the whole thing hinges on getting one judgement right: what counts as *the same line, changed* versus *a different line*.

Between two consecutive states the tool strips the common head and tail and runs an LCS diff over what is left. Each surviving line is then compared through a normalising key: SSA numbers are erased, block labels are erased, and what remains is the instruction's actual shape. So `%3 = load i64, ptr %2` becoming `%7 = load i64, ptr %5` is classified *renamed* — the same instruction after renumbering — while `%3 = load i64, ptr %2` becoming `%7 = getelementptr inbounds nuw %"class.jsonq::Lexer", ptr %0, i32 0, i32 3` is *rewritten*, and a line that was not there before is *created*.

That distinction is the difference between a history you can read and a list of every pass in the lane. Without it, every one of SROA's 28 runs and SimplifyCFG's 45 would claim whatever line it renumbered, and the chain would be noise. The chains are interned — pass names, events and history links are stored once and referenced — so the report ships the final state of the module as text rather than every intermediate state it walked through.

Instruction selection, without guessing

Correlating IR to Machine IR could have been done by matching instruction order and hoping. It isn't, because `llc` already leaves better evidence than that: each machine basic block is named after the IR block it came from, and machine operands that came from IR values carry `%ir.N` markers. The tool reads those names instead of inferring the correspondence, and reports coverage — `4/4 machine blocks traced · 6 value refs` — so a partial mapping is visibly partial.

Machine IR to assembly

Assembly is where machine instructions end up as text, and the printer does not preserve one-to-one ordering. The tool aligns the two by longest common subsequence over an opcode-compatibility predicate — `JCC_1` with `ja`, `SETCC` with `sete`, `CMOV` with `cmov` — which tolerates the assembler's spelling without pretending the two sequences are identical. What it cannot pair, it leaves unpaired and counts.

What LLVM computes, and what we compute

Worth being precise about. Every pass list, every diff, every line of IR and Machine IR, every timing and every analysis name comes from LLVM's own output. We

parse it, order it, join it, and lay it out. The blame chains, the ISEL and assembly correlation, the analysis graphs (dominators, dependence, loop nesting, the call graph), and the flow view's collapsing are ours — computed in Python, from the IR that `opt` and `llc` printed.

Two consequences. First, a bug in LLVM's dump is a bug in our report. Second, the report can be wrong about *interpretation* but never about *content*: the IR text in the diff is the text `opt` wrote, character for character.

Saying it once: the settings file

The flags are the interface. They are not the workflow. Nobody retypes `--load-pass-plugin ./libMBAAAdd.so --custom-pass mba-add --no-ai` every time they rebuild the report they are actually working on, and most of what you want to say is not a flag at all. So LLVM-Lens also reads its settings from a file.

It looks for `llvm-lens.yml` — or `.yml`, or `.llvm-lens.yml` — in the working directory and then in every parent, and falls back to `~/.config/llvm-lens/config.yml`; `LLVM_LENS_SETTINGS` or `--config` replaces the search entirely, and `--no-config` turns it off. Paths written in the file resolve against the file rather than your shell, so a config can be committed beside the code it analyses and behave the same from any directory. `output` is the one exception and stays relative to where you ran the command. Every key is optional, and a flag always wins over the file — the file is where you put what you already know, and the flag is for the one thing that is different today.

```
# examples/mba-add/llvm-lens.yml — the whole file
plugins:
  dir: .                # bare names resolve here, so no loader path needed
  pass: [libMBAAAdd.so]
custom-passes: [mba-add] # appended as function(mba-add), and badged
ai: false              # a report directory is meant to be shared
ui:
  changed-only: true    # the passes that changed nothing are noise
```

With that file in place, the entire command is `llvm-lens demo.ll`. The toolchain lives there too — which LLVM version you are on, where its binaries are, the target triple — along with the pipeline string, extra arguments for each of the three invocations, the timeout, which plugin kind you are loading, and whether source correlation is worth its extra `llc` run.

The `ui:` block is the part that saves the most, and it has no flag equivalent at all. It sets the report's own opening state: which pane and which view, which analysis graphs to draw, the split direction and ratio, which drawer tab is open, and whether the pass list starts filtered to the passes that changed something. A report you rebuild ten times a day opens ten times already pointed at the question you are asking, instead of at the default view of a pipeline you have read a hundred times.

Two details make it safe to leave lying around. An unknown key is refused with the nearest real key suggested, rather than silently ignored — the failure mode you would otherwise hit is a plausible-looking file that does nothing. And `llvm-lens --init-config` writes a fully commented starter with every key at the value it already has, so the file is discoverable without reading the source. The report carries the

config it read in its own metadata, so a report that surprises you still says where its settings came from.

Why the output is plain files

The report is `index.html` plus a `data/` directory of JSON chunks, with a copy of the frontend. Each chunk is written twice — as `.json` and as a `.js` that assigns the same object to a global — so the page works when opened straight off disk with `file://`, where `fetch` is not allowed. No server, no CORS, and a report you can zip and email. It also means the report has no runtime dependency on this project at all: it is a document.

06 Repo, team, and reproducing this

Everything in this document was produced by the code in the repository, on 18 September 2026, on a stock LLVM 22.

Running it

```
# needs python 3.10+ and llvm 22 tools on PATH
git clone https://github.com/manasghandat/LLVM-Lens.git
cd LLVM-Lens
pip install -e .
llvm-lens --sample --open # a report, no input file needed
llvm-lens yourfile.cpp --passes 'default<02>' -o report
```

Five samples ship with the package — `licm`, `register-pressure`, `side-channel`, `switch-lowering`, `vectorize` — each written so one behaviour is legible. `examples/json-query/` is a working JSON parser in seven translation units; `examples/mba-add/` is an out-of-tree pass plugin with the config file that wires it into the report.

Before you type that second command twice, run `llvm-lens --init-config`: it writes a commented `llvm-lens.yml` listing every setting at the value it already has, and from then on the pipeline, the toolchain, the plugins and the report's opening view come from the file instead of your shell history. Section 05 explains what goes in it; it is the difference between a tool you invoke and a tool that already knows what you are working on.

GPL-3.0. Built for SEGFAULT Compiler Hackathon 2026. The problem statement asked for an explainable, evidence-backed view of LLVM's optimization pipeline — which passes ran, what they changed, how significant each change was, and how it affected final codegen. That is what is in the repository, and what is in the figures above.